



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The state space is all the possible states in the problem. In my grid it is every (x, y) cell the agent can be in and the moves between them. Each state appears only once. The search tree is the structure the algorithm builds when it explores. Every node in the tree is a path from the start, so the same cell can appear many times in the tree if it is reached in different ways. So the state space is fixed and finite (144 cells for 12x12) but the search tree can get much bigger than that, even infinite if loops are not handled.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set solves the repeated states problem. Without it the search can go back to cells it already expanded and it will keep generating the same nodes again and again. This wastes time and can cause infinite loops. Adding the reached set changes the algorithm from tree search to graph search, so each state is expanded only once. If I removed it from my DFS agent, the agent would move into a cell and then expand its neighbours, which includes the cell it just came from. It would go back and forward between those cells and never finish. Even on the small grid it could run forever without reaching the food.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

In the simulation BFS walks straight to the closest pellet but DFS goes around the grid and takes long detours before it finally gets to the food. BFS expands the shallowest nodes first, so when it reaches the goal for the first time that is the smallest depth possible. In this grid every move has the same cost of 1, so the shallowest solution is also the cheapest one. That is why BFS is optimal here. DFS expands the deepest node first. It picks one direction and keeps going that way until it is blocked, then tries the next option. It returns the first path it finds, not the shortest, so the routes look winding and are not optimal.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS and UCS have space complexity $O(b^d)$. They keep all the nodes on the current level in the frontier at the same time, so the memory grows exponentially with depth. On a 1000x1000 grid the frontier and reached set could hold hundreds of thousands of nodes before food is found and the program would run out of memory. DFS has space complexity $O(bm)$ because it only keeps the current path from the root and the siblings that are not expanded yet. So its memory grows linearly with depth, not exponentially. The downside is that DFS time complexity is $O(b^m)$ where m can be much larger than d , and it is not optimal. But it would not run out of memory like BFS does.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING